

BLOQUE II • Calculadora BCD en VHDL

Memoria Final del Proyecto

Fases 1, 2 y 3 — Documento Completo

Diseño, implementación y verificación en VHDL-93

bcd_to_bin bin_to_bcd alu ctrl_tec displays controlador calculadora

Versión 1.0 — Junio 2026

- 📅 **Entrega Fase 1:** 6 de Mayo de 2026
- 📅 **Entrega Fase 2:** 14 de Mayo de 2026
- 📅 **Entrega Fase 3:** 28 de Mayo de 2026
- 📅 **Memoria Final:** 3 de Junio de 2026
- 🖨️ **Plataforma:** DECA MAX10 + conector XDECA



Miembros del Grupo

Lucía Llana Carmona	lucia.llana@alumnos.upm.es
María Moya Marcilla	maria.moya.marcilla@alumnos.upm.es
Pedro Gil Bolaños	pedro.gilb@alumnos.upm.es
Pedro Osma Moya	p.osma@alumnos.upm.es
Eduardo Muñoz de Maya	eduardo.mdemaya@alumnos.upm.es

Índice

Diseño Digital 2

Curso académico 2025–2026

BLOQUE II • Conversores de Código

Memoria del Proyecto Bloque II (Fase I)

Diseño, implementación y verificación en VHDL

bcd_to_bin $\longleftrightarrow$ bin_to_bcd

(Versión 1.0 — Mayo 2026)

 **Fase:** Bloque II (Fase I)

 **Fecha:** 6 Mayo de 2026



Miembros del Grupo

Lucía Llana Carmona	lucia.llana@alumnos.upm.es
María Moya Marcilla	maria.moya.marcilla@alumnos.upm.es
Pedro Gil Bolaños	pedro.gilb@alumnos.upm.es
Pedro Osma Moya	p.osma@alumnos.upm.es
Eduardo Muñoz de Maya	eduardo.mdemaya@alumnos.upm.es

Índice

1. Introducción y Objetivos

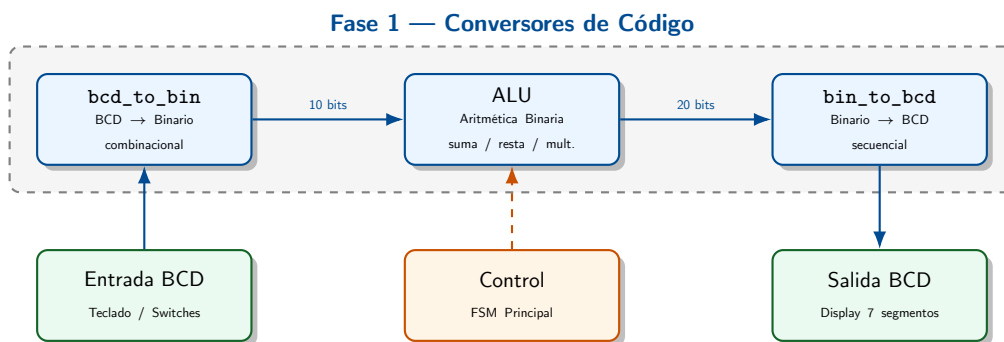
En esta primera fase el trabajo ha consistido en proponer el **diseño jerárquico** de la calculadora y en modelar, simular y depurar los dos conversores de código: **BCD a Binario** (`bcd_to_bin`) y **Binario a BCD** (`bin_to_bcd`).

Estos dos módulos son los más importantes de cara a la interfaz del sistema. El usuario introduce y recibe los datos en BCD (teclado y display de siete segmentos), pero toda la aritmética interna —suma, resta y multiplicación— se hace en binario en complemento a dos. Por eso necesitamos estos conversores: actúan de puente entre los dos dominios.

i **Todos los módulos** emplean `ieee.std_logic_unsigned`. Este paquete no soporta la multiplicación directa de vectores de longitud arbitraria, por lo que se recurre a desplazamientos (shifts) y sumas, que son la representación hardware natural de las potencias de dos.

1.1. Arquitectura jerárquica del sistema

La calculadora se estructura de forma jerárquica con los siguientes bloques principales:



Los bloques `bcd_to_bin` y `bin_to_bcd` son, por tanto, la frontera entre el dominio decimal del usuario y el binario interno. En esta fase nos hemos centrado en diseñarlos y verificarlos con sus respectivos testbenches.

2. Conversor BCD a Binario (`bcd_to_bin`)

2.1. Especificación

Tipo: Lógica combinacional pura (sin reloj).
Entrada: `bcd(11..0)` — tres nibbles: centenas `bcd[11:8]`, decenas `bcd[7:4]`, unidades `bcd[3:0]`.
Salida: `bin(9..0)` — valor binario en `[0, 999]`.

La conversión es simplemente:

$$\text{bin} = c \times 100 + d \times 10 + u$$

El máximo posible es $9 \times 100 + 9 \times 10 + 9 = 999$. Con 10 bits se pueden representar hasta $2^{10} - 1 = 1023$, así que no hay riesgo de desbordamiento en ningún caso válido.

2.2. Implementación mediante desplazamientos

Como no se puede usar multiplicación directa con `std_logic_unsigned`, hemos descompuesto los factores 100 y 10 como suma de potencias de dos y los hemos implementado con concatenaciones de ceros (equivalente a desplazamientos a la izquierda):

$$c \times 100 = c \times 64 + c \times 32 + c \times 4 = (c \ll 6) + (c \ll 5) + (c \ll 2)$$

$$d \times 10 = d \times 8 + d \times 2 = (d \ll 3) + (d \ll 1)$$

```

1  -- Extender los tres dígitos BCD a 10 bits cada uno
2  c <= "000000" & bcd(11 downto 8); -- centenas
3  d <= "000000" & bcd( 7 downto 4); -- decenas
4  u <= "000000" & bcd( 3 downto 0); -- unidades
5
6  -- c * 100 = c*64 + c*32 + c*4 (64+32+4 = 100)
7  c100 <= (c(3 downto 0) & "000000") -- c * 64 (shift 6)
8  + (c(4 downto 0) & "00000") -- c * 32 (shift 5)
9  + (c(7 downto 0) & "00"); -- c * 4 (shift 2)
10
11 -- d * 10 = d*8 + d*2 (8+2 = 10)
12 d10 <= (d(6 downto 0) & "000") -- d * 8 (shift 3)
13 + (d(8 downto 0) & '0'); -- d * 2 (shift 1)
14
15 bin <= c100 + d10 + u;

```

Listado 1: Núcleo de bcd_to_bin.vhd — multiplicación por desplazamientos

⚠️ Anchos de bits: Cada concatenación produce exactamente 10 bits, igual que la señal destino. Por ejemplo, `c(3 downto 0) & "000000"` produce $4 + 6 = 10$ bits. Con `std_logic_unsigned` las sumas operan en ese ancho fijo, descartando cualquier carry sin riesgo de desbordamiento silencioso.

2.3. Verificación

El testbench `tb_bcd_to_bin` simplemente va cambiando la entrada cada 20 ns sin necesidad de reloj, ya que el módulo es puramente combinacional. Los casos cubren los extremos y algunos valores intermedios:

Cuadro 1: Casos de prueba del testbench `tb_bcd_to_bin`

Entrada BCD (hex)	Salida binaria (decimal)	Propósito
0x000	0	Cero absoluto
0x010	10	Solo decenas
0x100	100	Solo centenas
0x123	123	Valor mixto representativo
0x999	999	Máximo absoluto

3. Conversor Binario a BCD (`bin_to_bcd`)

3.1. Especificación

Tipo: Lógica secuencial — FSM de tres estados.
Entradas: `clk` (50 MHz), `nRst` (reset activo bajo), `start`, `bin(N_BITS-1..0)`.
Salidas: `done` (pulso de un ciclo), `bcd(23..0)` — seis dígitos BCD.
Genérico: `N_BITS = 20` $\Rightarrow$ rango $[0, 2^{20} - 1] = [0, 1\,048\,575]$.

La calculadora tiene que poder mostrar resultados de hasta $999 \times 999 = 998\,001$, así que el conversor trabaja con 20 bits de entrada y produce 6 dígitos BCD (24 bits). Hacer esto con lógica combinacional pura sería inviable, así que hemos optado por una **arquitectura secuencial iterativa**.

3.2. Algoritmo: Suma de pesos (Horner en BCD)

Hemos implementado el algoritmo de suma de pesos, que construye el resultado BCD procesando el número binario bit a bit de más significativo a menos, en exactamente $N_{\text{BITS}} = 20$ ciclos de reloj. La idea es aplicar el *método de Horner* pero realizando la aritmética directamente en BCD:

$$\text{acc} \leftarrow 2 \cdot \text{acc} + b_i \quad (\text{aritmética BCD}), \quad i = N - 1, N - 2, \dots, 0$$

Cada ciclo hace lo siguiente:

Paso 1. Doblar en BCD: para cada dígito BCD del acumulador se calcula $tmp = 2 \cdot d_k + c_{entrada}$ (5 bits, rango 0–19). Si $tmp \geq 10$, se resta 10 al resultado y se genera $c_{salida} = 1$ que se propaga al dígito superior.

Paso 2. Sumar el bit entrante: el bit MSB de `bin_reg` actúa como $c_{entrada}$ del dígito de unidades (vale 0 o 1, por lo que nunca causa desbordamiento por sí solo).

💡 **Ejemplo (8 bits, $10011101_2 = 157_{10}$):** $Peso_{128}=1$, $Peso_{64}=2 \times 1 + 0 = 2$, $Peso_{32}=2 \times 2 + 0 = 4$, $Peso_{16}=2 \times 4 + 1 = 9$, $Peso_8=2 \times 9 + 1 = 19$, $Peso_4=2 \times 19 + 1 = 39$, $Peso_2=2 \times 39 + 0 = 78$, $Peso_1=2 \times 78 + 1 = 157$ (todo en BCD)

```

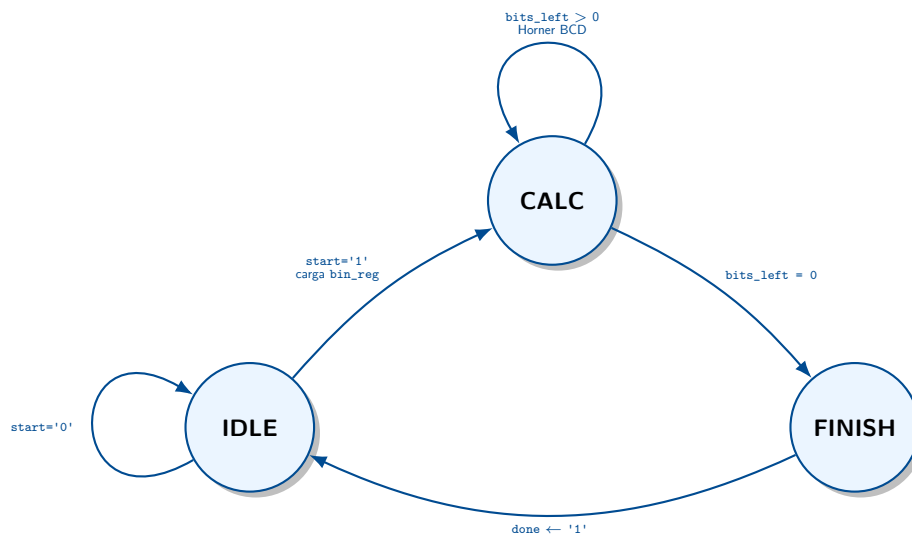
1  -- tmp = 2*digito + carry_in (5 bits, rango 0..19)
2  -- Si tmp >= 10: resultado = tmp(3:0) + 6 (resta 10 = suma 6 en nibble)
3  --      carry_out = 1
4  c := bin_reg(N_BITS-1); -- bit entrante -> carry inicial del digito 0
5  tmp := ('0' & acc_bcd(3 downto 0)) + ('0' & acc_bcd(3 downto 0)) + ("0000" & c);
6  if tmp >= "01010" then n0 := tmp(3 downto 0) + "0110"; c := '1';
7  else n0 := tmp(3 downto 0); c := '0'; end if;
8  -- (idem para n1..n5: decenas, centenas, miles, decenas de miles, centenas de miles)
9
10 acc_bcd <= n5 & n4 & n3 & n2 & n1 & n0;
11 bin_reg <= bin_reg(N_BITS-2 downto 0) & '0'; -- avanzar registro binario
12 bits_left <= bits_left - 1;

```

Listado 2: Núcleo del algoritmo de suma de pesos en `bin_to_bcd.vhd`

3.3. Máquina de Estados (FSM)

El control secuencial se implementa mediante una FSM de tres estados:



Cuadro 2: Estados de la FSM del conversor `bin_to_bcd`

Estado	Descripción	Acción sobre señales
IDLE	Espera activa de <code>start='1'</code> . Carga <code>bin_reg ← bin</code> .	<code>done ← '0'</code>
CALC	Ejecuta un paso Horner BCD por ciclo durante <code>N_BITS</code> ciclos.	<code>acc_bcd</code> , <code>bin_reg</code> actualizados
FINISH	Pulso de fin; retorna inmediatamente a IDLE.	<code>done ← '1'</code>

3.4. Verificación

El testbench `tb_bin_to_bcd` genera un reloj de 50 MHz (periodo 20 ns). Nos encontramos con problemas de condiciones de carrera al detectar `done='1'`, así que lo resolvimos con un procedimiento auxiliar (`run_test`) que alinea todo con flancos de reloj. La simulación termina automáticamente mediante `assert false severity failure`, por lo que basta con ejecutar `run -all`.

```

1  procedure run_test(val : std_logic_vector(19 downto 0)) is
2  begin
3  wait until clk'event and clk = '1'; -- alinear con el reloj antes de asignar
4  bin <= val;
5  start <= '1';
6  wait until clk'event and clk = '1'; -- IDLE detecta start; done pasa a '0'
7  start <= '0';
8  wait until done = '1';              -- esperar fin de la conversión (20 ciclos)
9  wait for clk_period * 2;            -- margen para visualizar en waveform
10 end procedure;
```

Listado 3: Procedimiento auxiliar `run_test` del testbench secuencial

Cuadro 3: Casos de prueba del testbench `tb_bin_to_bcd`

Binario (decimal)	BCD esperado (hex)	Propósito
0	0x000000	Cero absoluto
1	0x000001	Unidad mínima
9	0x000009	Máximo de un dígito
10	0x000010	Decena exacta
100	0x000100	Centena exacta
255	0x000255	Potencia de 2 menos 1
999	0x000999	Máximo operando de entrada
1 000	0x001000	Paso de millar
9 999	0x009999	Máximo de cuatro dígitos
999 999	0x999999	Máximo BCD representable (6 dígitos)

Nota: el valor máximo operacional de la calculadora es $999 \times 999 = 998\,001$. El último caso de prueba utiliza 999 999 porque es el máximo que los 24 bits BCD de salida pueden representar, ofreciendo así una verificación de frontera más exigente.

4. Conclusiones

En esta primera fase hemos diseñado, implementado y verificado los dos conversores de código que forman la base del proyecto:

- `bcd_to_bin`: completamente combinacional. Hemos resuelto la multiplicación por 100 y por 10 mediante desplazamientos y sumas, sin usar multiplicadores, lo que lo hace compatible con `std_logic_unsigned`. La latencia es nula (solo depende de la propagación de puertas).
- `bin_to_bcd`: secuencial con FSM de tres estados (IDLE / CALC / FINISH). El algoritmo de suma de pesos (Horner en BCD) tarda exactamente 20 ciclos de reloj, lo que es más que suficiente para los valores que puede generar la calculadora ($999 \times 999 = 998\,001$ como máximo).

Todos los casos de prueba han dado el resultado esperado, incluyendo el cero, los valores de frontera y algunos intermedios. Los módulos quedan listos para integrarse en las siguientes fases.

➔ **Próxima fase:** En la Fase 2 habrá que diseñar la ALU: suma, resta y multiplicación de operandos con signo en complemento a dos (10 bits), con detección de desbordamiento mediante la señal `overflow`.

A. Investigación y conocimientos adquiridos durante el desarrollo

Durante la realización de esta fase surgieron diversas dificultades técnicas cuya resolución requirió investigación adicional fuera del contenido impartido en clase. Se documentan a continuación los aspectos más relevantes.

A.1. Algoritmo de suma de pesos (Horner en BCD)

Para la conversión binario a BCD se buscaba un método que no requiriese divisiones ni operaciones módulo, ya que no son sintetizables directamente con `std_logic_unsigned` y resultan costosas en área hardware.

El algoritmo elegido se basa en la **representación polinómica** de un número binario, evaluada mediante el método de Horner:

$$\text{bin} = b_{N-1} \cdot 2^{N-1} + \dots + b_1 \cdot 2 + b_0 = (\dots((b_{N-1}) \cdot 2 + b_{N-2}) \cdot 2 + \dots) \cdot 2 + b_0$$

La clave es que esta evaluación se realiza *íntegramente en aritmética BCD*: en cada paso se dobla el acumulador BCD y se suma el siguiente bit. “Doblar en BCD” significa multiplicar cada dígito por 2, y si el resultado supera 9 se resta 10 y se propaga un carry al dígito superior.

💡 **Intuición del doblado BCD:** Dado un dígito $d \in [0, 9]$, al doblar $d' = 2d + c_{in}$ (rango 0–19). Si $d' \geq 10$: el resultado decimal es $d' - 10$ con $c_{out} = 1$ hacia el dígito superior. En hardware se implementa como: si $d' \geq 10$, suma 6 al nibble (porque $16 - 10 = 6$, lo que equivale a restar 10 aprovechando el desbordamiento natural de 4 bits) y activa el carry.

La comprensión del algoritmo requirió trazar paso a paso el ejemplo de 8 bits con el número $10011101_2 = 157_{10}$, verificando que en cada iteración el acumulador BCD es exactamente el peso del bit ya procesado. Una vez comprendido, la implementación en VHDL resultó directa.

A.2. Multiplicación sin operador de multiplicación

Para el conversor BCD a binario fue necesario calcular $c \times 100$ y $d \times 10$ sin disponer de un operador de multiplicación genérico. La solución se obtuvo descomponiendo cada factor como suma de potencias de dos:

$$100 = 2^6 + 2^5 + 2^2, \quad 10 = 2^3 + 2^1$$

Multiplicar por una potencia de dos equivale a un desplazamiento a la izquierda, que en VHDL se implementa mediante concatenación de ceros, sin coste alguno en lógica combinatorial. Este enfoque no era conocido de antemano y se dedujo a partir del análisis de la representación binaria de los factores.

BLOQUE II • ALU, Teclado y Displays

Memoria del Proyecto

Bloque II — Fase 2

Diseño, implementación y verificación en VHDL-93

alu clk_div timer ctrl_tec interfaz_teclado displays

Versión 1.0 — Mayo 2026

📁 **Fase:** Bloque II — Fase 2

📅 **Entrega:** 14 de Mayo de 2026

🔌 **Plataforma:** DECA MAX10 + conector XDECA

🕒 **Reloj:** 50 MHz

👥 Miembros del Grupo

Lucía Llana Carmona	lucia.llana@alumnos.upm.es
María Moya Marcilla	maria.moya.marcilla@alumnos.upm.es
Pedro Gil Bolaños	pedro.gilb@alumnos.upm.es
Pedro Osma Moya	p.osma@alumnos.upm.es
Eduardo Muñoz de Maya	eduardo.mdemaya@alumnos.upm.es

Índice

B. Introducción y objetivos

En esta segunda fase se han diseñado, implementado y verificado los bloques intermedios de la calculadora: la **unidad aritmético-lógica (ALU)**, el **sistema de temporización**, el **controlador de teclado matricial** y el **controlador de displays multiplexados**. Estos módulos constituyen el núcleo de la *capa de interfaz* del sistema.

El objetivo de la fase es disponer de un camino de datos completamente funcional y simulable que pueda integrarse directamente con el controlador (FSM) y los conversores de código desarrollados en la Fase 1.

❗ Criterio de codificación adoptado en todo el proyecto: todos los módulos RTL emplean **señales** en lugar de variables. Una señal modela directamente un cable o un registro físico: su valor solo se actualiza al término del proceso (equivalente al flanco de reloj), igual que ocurre en el circuito real. Por el contrario, una variable se actualiza de forma inmediata dentro del proceso—comportamiento propio del software—lo que dificulta la correspondencia con el hardware y oculta la señal al simulador (no aparece en las formas de onda). Este criterio se justifica con detalle en el Apéndice ??.

C. Arquitectura global del sistema

La figura C muestra la jerarquía completa con todos los módulos, señales de interconexión y agrupación por fases de desarrollo.

Arquitectura jerárquica completa de la calculadora.

C.1. Señales de interconexión principales

Cuadro 4: Señales clave entre módulos

Señal	Ancho	De	A
tecla[3:0]	4 b	ctrl_tec	controlador
tecla_pulsada	1 b	ctrl_tec	controlador
op1_bcd[11:0]	12 b	controlador	bcd_to_bin, displays
op2_bcd[11:0]	12 b	controlador	bcd_to_bin, displays
op1_bin[10:0]	11 b	lógica inline C2	alu
op2_bin[10:0]	11 b	lógica inline C2	alu
res[19:0]	20 b	alu	bin_to_bcd
res_sgn	1 b	alu	controlador, displays
bcd[23:0]	24 b	bin_to_bcd	displays
done	1 b	bin_to_bcd	controlador
tic_1ms	1 b	timer	displays
pres[1:0]	2 b	controlador	displays

D. Unidad Aritmético-Lógica (alu.vhd)

D.1. Especificación

Tipo: Lógica combinacional (sin reloj).

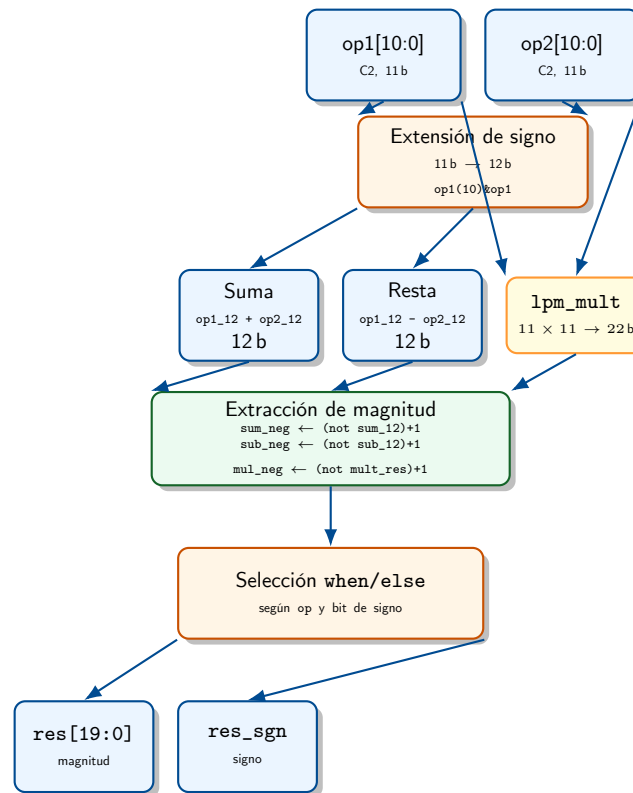
Entradas: op1[10:0], op2[10:0] en complemento a 2, rango $[-999, +999]$; op[1:0]: "00"=suma, "01"=resta, "10"=multiplicación.

Salidas: res[19:0] magnitud del resultado; res_sgn signo ('1' = negativo); overflow (fijado a '0': los operandos ± 999 no pueden desbordar 20 bits).

Rango máx. $999 \times 999 = 998\,001 < 2^{20}$.

D.2. Camino de datos

La figura siguiente muestra el camino de datos interno de la ALU.



D.3. Implementación en VHDL

Los operandos se extienden de 11 a 12 bits replicando el bit de signo (MSB), lo que permite realizar suma y resta directamente en aritmética de complemento a 2 con un bit extra que actúa como indicador de signo del resultado:

```

1  -- Extencion de signo: 11 -> 12 bits (replica op(10) en la posicion 11)
2  op1_12 <= op1(10) & op1;
3  op2_12 <= op2(10) & op2;
4
5  -- Suma y resta en complemento a 2 (12 bits)
6  sum_12 <= op1_12 + op2_12;
7  sub_12 <= op1_12 - op2_12;

```

Listado 4: Extensión de signo y operaciones aritméticas en alu.vhd

Para la multiplicación se instancia el componente lpm_mult, que es el modelo de simulación de una IP de Altera (multiplicador con signo de $11 \times 11 \rightarrow 22$ bits):

```

1  U_MULT: lpm_mult
2      port map(
3      dataa => op1,
4      datab => op2,
5      result => mult_res    -- 22 bits, complemento a 2
6  );

```

Listado 5: Instanciación del multiplicador IP en alu.vhd

La extracción de la magnitud y el signo se realiza íntegramente con **asignaciones concurrentes** when/else, sin ningún proceso ni variable. Primero se calculan los complementos a 2 de los resultados posibles (para tener la magnitud en el caso negativo), y luego se selecciona:

```

1  -- Complementos a 2: magnitud cuando el resultado es negativo
2  sum_neg <= (not sum_12) + 1;
3  sub_neg <= (not sub_12) + 1;
4  mul_neg <= (not mult_res) + 1;
5
6  -- Seleccion de magnitud segun operacion y bit de signo del resultado
7  res <= "00000000" & sum_neg    when op="00" and sum_12(11) = '1' else
8         "00000000" & sum_12    when op="00"                        else
9         "00000000" & sub_neg    when op="01" and sub_12(11) = '1' else

```

```

10      "00000000" & sub_12      when op="01"      else
11      mul_neg(19 downto 0)      when op="10" and mult_res(21)='1' else
12      mult_res(19 downto 0);
13
14  -- Signo: '1' cuando el resultado es negativo
15  res_sgn <= '1' when (op="00" and sum_12(11) = '1') or
16                    (op="01" and sub_12(11) = '1') or
17                    (op="10" and mult_res(21)='1') else '0';

```

Listado 6: Extracción de magnitud sin variables ni proceso en alu.vhd

💡 **¿Por qué when/else y no un proceso?** Porque la ALU es combinacional: no hay reloj ni estado. Un proceso con lista de sensibilidad sería equivalente, pero las asignaciones concurrentes expresan directamente la intención — “este cable vale X o Y dependiendo de la condición” — sin necesitar variables intermedias.

D.4. Verificación: tb_alu.vhd

El testbench es puramente combinacional (sin reloj). Cambia las entradas cada 20 ns y comprueba el resultado mediante la función auxiliar check, que genera un report de error si la magnitud o el signo no coinciden con los esperados:

```

1  procedure check(
2      tag      : in string;
3      got_mag  : in std_logic_vector(19 downto 0);
4      got_sgn  : in std_logic;
5      exp_mag  : in integer;
6      exp_sgn  : in std_logic
7  ) is
8  begin
9      assert got_sgn = exp_sgn
10         report tag & ": signo incorrecto." severity error;
11      assert got_mag = exp_mag
12         report tag & ": magnitud incorrecta." severity error;
13  end procedure;

```

Listado 7: Procedimiento check del testbench tb_alu

Cuadro 5: Casos de prueba del testbench tb_alu (10 casos)

op1	op2	Operación	Magnitud esperada	Signo	Propósito
+100	+200	suma	300	+	Suma positivos
-100	-200	suma	300	-	Suma negativos
-100	+200	suma	100	+	Suma signos distintos
+300	+100	resta	200	+	Resta normal
+100	+300	resta	200	-	Resultado negativo
-100	+200	resta	300	-	Resta con negativo
+12	+12	mult	144	+	Producto simple
-3	+5	mult	15	-	Producto negativo
+999	+999	mult	998 001	+	Valor máximo posible
0	+999	mult	0	+	Cero en multiplicación

La simulación termina con **Errors: 0, Warnings: 0** en ModelSim 10.4d con VHDL-93.

E. Temporización: clk_div.vhd y timer.vhd

E.1. Divisor de reloj genérico (clk_div)

Genera un pulso tic de **exactamente un ciclo de reloj** de anchura cada DIV+1 ciclos. El genérico DIV permite instanciar el mismo módulo para distintos periodos:

```

1 process(clk, nRst)
2 begin
3     if nRst = '0' then
4         cnt <= 0;
5         tic_reg <= '0';
6     elsif clk'event and clk = '1' then
7         tic_reg <= '0';           -- por defecto pulso bajo
8         if cnt = DIV then
9             tic_reg <= '1';       -- pulso de 1 ciclo
10            cnt <= 0;
11        else
12            cnt <= cnt + 1;
13        end if;
14    end if;
15 end process;
16 tic <= tic_reg;                 -- asignacion concurrente fuera del proceso

```

Listado 8: Implementación de clk_div.vhd

Cuadro 6: Instancias de clk_div en el proyecto

Uso	DIV	Periodo resultante
Tic teclado (interfaz_teclado)	249 999	5 ms a 50 MHz
Simulación (genérico reducido)	4	100 ns

E.2. Temporizador doble (timer)

Genera dos pulsos independientes a partir del mismo reloj de 50 MHz: tic_1ms para la multiplexación de displays y tic_5ms como referencia temporal adicional.

```

1 -- Tic cada 1 ms
2 if cnt_1ms = DIV_1ms then
3     t1ms_reg <= '1';
4     cnt_1ms <= 0;
5     -- Tic cada 5 ms (cada 5 tics de 1 ms)
6     if cnt_5ms = DIV_5ms then
7         t5ms_reg <= '1';
8         cnt_5ms <= 0;
9     else
10        cnt_5ms <= cnt_5ms + 1;
11    end if;
12 else
13    cnt_1ms <= cnt_1ms + 1;
14 end if;

```

Listado 9: Generación de tic_1ms y tic_5ms en timer.vhd

Los dos contadores son completamente independientes en el proceso y se resetean con el mismo nRst activo bajo.

F. Controlador de teclado (ctrl_tec.vhd)

F.1. Descripción general y especificación de puertos

El módulo gestiona un teclado matricial 4×4 conectado al conector XDECA. La interfaz utiliza buses de 4 bits en lugar de señales individuales, lo que simplifica el cableado en la jerarquía superior:

Puerto	Dirección	Descripción
clk	entrada	Reloj 50 MHz
nRst	entrada	Reset activo bajo
tic	entrada	Pulso 5 ms (de clk_div)
columna[3:0]	entrada	Columnas activo bajo
fila[3:0]	buffer	Filas de escaneo, activo bajo
tecla[3:0]	buffer	Código hex de la última tecla
tecla_pulsada	buffer	Pulso 1 ciclo al soltar la tecla
pulso_largo	buffer	Activo tras 2s mantenida

Genérico: TICS_2s = 400 (400 tics $\times$ 5 ms = 2 s a 50 MHz).

F.2. Mapa del teclado

La distribución física del teclado 4 $\times$ 4 se codifica en complemento one-hot activo bajo para filas y columnas:

Cuadro 7: Distribución de teclas: código hex asignado a cada posición

fila	col "1110"	col "1101"	col "1011"	col "0111"
"1110" fila 0	1	2	3	F
"1101" fila 1	4	5	6	E (×)
"1011" fila 2	7	8	9	D (−)
"0111" fila 3	A (+)	0	B (=)	C (±)

F.3. Mecanismo de escaneo de filas

El módulo activa una fila a la vez poniendo su bit a '0' (activo bajo), manteniendo las demás a '1'. En cada tic de 5 ms rota el patrón one-hot hacia la izquierda:

fila : "1110" $\rightarrow$ "1101" $\rightarrow$ "1011" $\rightarrow$ "0111" $\rightarrow$ "1110" $\rightarrow$...

```

1 process(clk, nRst)
2 begin
3     if nRst = '0' then
4         fila <= (others => '1'); -- todas inactivas en reset
5     elsif clk'event and clk = '1' then
6         if tic = '1' then
7             if ena_muestreo = '1' then
8                 if fila = X"F" then
9                     fila <= "1110"; -- inicio del escaneo
10                else
11                    fila <= fila(2 downto 0) & fila(3); -- rotacion izquierda
12                end if;
13            end if;
14        end if;
15    end if;
16 end process;

```

Listado 10: Rotación one-hot de filas en ctrl_tec.vhd

El escaneo se pausa (ena_muestreo = '0') mientras se detecta actividad en las columnas, para evitar capturar una columna en medio del cambio de fila.

F.4. Antirebote integrado

No se usa ningún módulo externo de antirebote. La columna leída se registra en dos etapas:

1. col_reb: muestreo síncrono de columna en cada ciclo de reloj (elimina los metaestables por cruce de dominios de reloj).
2. col_ok: se actualiza con el valor de col_reb solo en cada tic de 5 ms, garantizando que el valor lleva al menos 5 ms estable antes de ser aceptado.


```

1 process(clk, nRst)
2 begin
3     if nRst = '0' then
4         col_reb <= (others => '1');
5         col_ok  <= (others => '1');
6     elsif clk'event and clk = '1' then
7         col_reb <= columna;          -- etapa 1: sincronizar
8         if tic = '1' then
9             col_ok <= col_reb;        -- etapa 2: validar cada 5 ms
10        end if;
11    end if;
12 end process;

```

Listado 11: Antirebote de dos etapas en ctrl_tec.vhd

F.5. Detección de pulsación corta y larga

La señal interna ctrl_teclea_pulsada indica si hay alguna columna activa en col_ok. La lógica de detección es la siguiente:

- **Pulsación corta** (tecla_pulsada): pulso de 1 ciclo generado en el *flanco de bajada* de ctrl_teclea_pulsada (es decir, cuando se suelta la tecla), siempre que no haya habido pulsación larga:

```

tecla_pulsada <= '1' when pulso_hab_teclea = '1'
                  and ctrl_teclea_pulsada = '0'
                  and pulso_hab_largo    = '0'
                  else '0';

```

- **Pulsación larga** (pulso_largo): se activa cuando la tecla lleva pulsada $TICS_{2s} = 400 \text{ tics} \times 5 \text{ ms} = 2 \text{ s}$. En ese caso se suprime el evento de *release* para evitar registrar una pulsación corta fantasma.

❗ **¿Por qué detectar en el release y no en el press?** Detectar en la pulsación produce problemas si el usuario mantiene la tecla más de lo necesario (se registran múltiples pulsaciones). Detectar en el release garantiza exactamente un evento por pulsación, y permite además distinguir pulsaciones cortas de largas antes de generar el evento.

F.6. Decodificador fila × columna

El decodificador es un registro síncrono con reset que evalúa la combinación de fila y col_ok y guarda el código de tecla en tecla_aux. El resultado se transfiere a la salida tecla únicamente en el momento de la pulsación (corta o larga):

```

1 process(clk, nRst)
2 begin
3     if nRst = '0' then
4         tecla_aux <= (others => '0');
5     elsif clk'event and clk = '1' then
6         if fila="1110" and col_ok="1110" then tecla_aux <= X"1";
7         elsif fila="1110" and col_ok="1101" then tecla_aux <= X"2";
8         -- ... (16 combinaciones en total)
9         elsif fila="0111" and col_ok="0111" then tecla_aux <= X"C";
10        end if;
11    end if;
12 end process;

```

Listado 12: Decodificador fila/columna (fragmento) en ctrl_tec.vhd

F.7. Jerarquía: interfaz_teclado.vhd

El módulo interfaz_teclado agrupa clk_div y ctrl_tec en un único bloque con la interfaz estándar del sistema. Los puertos de tipo buffer de ctrl_tec se conectan a través de señales internas para cumplir la restricción de VHDL-93 (no se puede conectar un puerto buffer directamente a un puerto out del módulo padre):

```

1 signal fila_s, tec_s : std_logic_vector(3 downto 0);
2 signal tp_s, pl_s    : std_logic;
3
4 U_DIV: entity work.clk_div(rtl)
5     generic map(DIV => DIV_TEC)
6     port map(clk => clk, nRst => nRst, tic => tic_s);
7
8 U_CTRL: entity work.ctrl_tec(rtl)
9     generic map(TICS_2s => TICS_2s)
10    port map(
11        clk => clk, nRst => nRst, tic => tic_s,
12        columna => columna, fila => fila_s,
13        tecla_pulsada => tp_s, tecla => tec_s,
14        pulso_largo => pl_s
15    );
16
17 -- Señales intermedias -> salidas del modulo padre
18 fila <= fila_s; tecla <= tec_s;
19 tecla_pulsada <= tp_s; pulso_largo <= pl_s;

```

Listado 13: Integración en interfaz_teclado.vhd

G. Controlador de displays (displays.vhd)

G.1. Especificación

Hardware:	8 displays de 7 segmentos, cátodo común, activo alto (DECA + XDECA).
Multiplexación:	Registro mux_reg one-hot activo bajo; rota en cada tic_1ms. Frecuencia de refresco: $1000\text{ Hz}/8 = 125\text{ Hz}$ (sin parpadeo visible).
Formato:	disp[7:0]: bits <i>dp, a, b, c, d, e, f, g</i> (bit 7 = punto decimal, bit 0 = g).
Selector pres[1:0]:	"00" = op1 en D4-D1; "01" = op2 en D4-D1; "10" = resultado en D8-D1.

G.2. Multiplexación

El registro mux_reg es un one-hot activo bajo que rota hacia la izquierda en cada tic_1ms, activando un display diferente en cada ciclo de refresco:

```

1 process(clk, nRst)
2 begin
3     if nRst = '0' then
4         mux_reg <= "1111110"; -- display 1 activo (bit 0 = '0')
5     elsif clk'event and clk = '1' then
6         if tic_1ms = '1' then
7             mux_reg <= mux_reg(6 downto 0) & mux_reg(7); -- rotacion izq
8         end if;
9     end if;
10 end process;

```

Listado 14: Registro de multiplexación en displays.vhd

G.3. Cálculo de los 8 dígitos a mostrar

Los 8 dígitos se calculan combinatorialmente como señales concurrentes sig_d1 ...sig_d8, sin variables ni proceso. Cada señal usa una cadena when/else que considera el modo pres y aplica la supresión de ceros a la izquierda:

```

1 -- D2 (decenas): blanco si tanto centenas como decenas son cero
2 sig_d2 <= x"E" when pres="00" and op1_bcd(11 downto 4) = x"00" else
3     op1_bcd(7 downto 4) when pres="00" else
4     x"E" when pres="01" and op2_bcd(11 downto 4) = x"00" else
5     op2_bcd(7 downto 4) when pres="01" else
6     x"E" when res_bcd(23 downto 4) = x"0000" else

```